

6. Evidencia de Machine Learning

Resumen del Módulo ML

El módulo de Machine Learning (`apps/ml/services/predictor.py`) permite entrenar un modelo de clasificación supervisada y realizar predicciones en vivo sobre el nivel de riesgo clínico de pacientes, utilizando **Random Forest Classifier** de Scikit-learn.

1. Dataset Entrenado

Fuente de Datos

- **Origen:** 1800 registros clínicos preprocesados por el pipeline ETL
- **Tabla:** `etl_pacienteclinico` en SQLite
- **Preprocesamiento:** Todos los datos han pasado por el proceso de limpieza ETL (sin nulos, sin outliers, sin duplicados)

Variables Predictoras (Features)

#	Variable	Tipo	Descripción
1	IMC	Float	Índice de Masa Corporal (peso/altura²)
2	edad	Integer	Edad del paciente en años
3	glucosa	Float	Nivel de glucosa en mg/dL
4	colesterol	Float	Nivel de colesterol en mg/dL
5	presion_sistolica	Integer	Presión arterial sistólica (mmHg)

6	presion_diastolica	Integer	Presión arterial diastólica (mmHg)
7	frecuencia_cardiaca	Integer	Frecuencia cardíaca (bpm)
8	fumador	Boolean (0/1)	Si el paciente fuma o no

Variable Objetivo (Target)

- **Campo:** riesgo_enfermedad
- **Clases (4 niveles):**

Clase	Valor Numérico	Descripción
Bajo	0	Paciente sin factores de riesgo significativos
Medio	1	Factores de riesgo moderados (IMC > 25, fumador, colesterol > 240)
Alto	2	Valores clínicos elevados (presión > 140, glucosa > 200, IMC > 35)
Crítico	3	Valores extremos (presión > 180, glucosa > 300, saturación < 85%)

Distribución del Target en el Dataset

Según los datos procesados por el ETL, la distribución de riesgo entre los 1800 pacientes es:

Riesgo	Cantidad Aproximada
Bajo	~25-30%
Medio	~35-40%

Alto	~20-25%
Crítico	~10-15%

Nota: Los valores exactos pueden consultarse en el endpoint `/api/analytics/segmentacion/`

2. Algoritmo de Machine Learning

Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100, # 100 árboles de decisión
    random_state=42   # Semilla para reproducibilidad
)
```

¿Por qué Random Forest?

- Excelente manejo de variables categóricas y numéricas combinadas
- Robusto frente al sobreajuste (overfitting)
- Capaz de capturar relaciones no lineales entre variables clínicas
- Proporciona importancia de características y probabilidades
- Buen rendimiento con datasets de tamaño moderado (1800 registros)

Pipeline de Entrenamiento

```
1. Consultar todos los registros de PacienteClinico
2. Convertir a DataFrame de Pandas
3. Mapear riesgo_enfermedad → valores numéricos (0-3)
4. Seleccionar las 8 features + target
5. Dividir datos: 80% entrenamiento, 20% prueba
6. Entrenar RandomForestClassifier
7. Evaluar con métricas de clasificación multiclase
8. Guardar modelo serializado en modelo_entrenado.pkl
```

3. Métricas de Rendimiento

Las métricas se calculan sobre el conjunto de prueba (20% de los 1800 registros $\approx$ 360 pacientes).

Definición de Métricas

Métrica	Fórmula	Interpretación
Accuracy	$(VP + VN) / Total$	Proporción de predicciones correctas sobre el total
Precision	$VP / (VP + FP)$	De los que el modelo predijo como X, qué proporción realmente es X
Recall	$VP / (VP + FN)$	De los que realmente son X, qué proporción identificó el modelo
F1-Score	$2 * (P * R) / (P + R)$	Media armónica de precisión y recall

Resultados Obtenidos

Métrica	Valor	Porcentaje
Accuracy	0.6389	63.89%
Precision	0.6394	63.94%
Recall	0.6389	63.89%
F1-Score	0.6347	63.47%

Matriz de Confusión

	Predicho			
	Bajo	Medio	Alto	Crítico
Real Bajo	[XX	XX	XX	XX]
Medio	[XX	XX	XX	XX]
Alto	[XX	XX	XX	XX]
Crítico	[XX	XX	XX	XX]

Interpretación de Resultados

- **Accuracy del 64%:** El modelo clasifica correctamente aproximadamente 2 de cada 3 pacientes. Es un rendimiento aceptable para una primera iteración considerando la naturaleza multiclase (4 categorías).

- **Precision y Recall balanceados:** Indican que el modelo no está sesgado hacia ninguna clase en particular.
- **Desafíos identificados:**
 - La distinción entre riesgo "Medio" y "Alto" es compleja porque los umbrales clínicos tienen varianzas pequeñas
 - Algunas categorías pueden tener menos representación (desequilibrio de clases)
 - Con solo 8 features numéricas, hay margen para incorporar más variables

Recomendaciones para Mejora

1. **Aumentar el dataset:** 1800 registros es un volumen moderado; más datos mejorarían la generalización
2. **Ajuste de hiperparámetros:** Optimizar `n_estimators`, `max_depth`, `min_samples_split`
3. **Balanceo de clases:** Usar SMOTE o `class_weight` para manejar clases desbalanceadas
4. **Ingeniería de features:** Crear nuevas variables como ratios (glucosa/colesterol), interacciones
5. **Probar otros algoritmos:** XGBoost, Gradient Boosting, o redes neuronales simples
6. **Validación cruzada:** Usar k-fold cross-validation para evaluación más robusta

4. Resultados y Uso Práctico

Modelo Persistido

El modelo entrenado se guarda en:

- **Archivo:** `backend/apps/ml/modelo_entrenado.pkl`
- **Formato:** Joblib (serialización binaria de Python)
- **Persistencia:** El modelo permanece disponible entre reinicios del servidor

Endpoint de Predicción

POST `/api/ml/predecir/`

Ejemplo de Request:

```
{  
  "edad": 65,  
}
```

```
"IMC": 32.5,  
"glucosa": 180,  
"colesterol": 250,  
"presion_sistolica": 160,  
"presion_diastolica": 95,  
"frecuencia_cardiaca": 85,  
"fumador": 1  
}
```

Ejemplo de Response:

```
{  
  "riesgo_predicho": "Alto",  
  "probabilidad": 0.7843,  
  "distribucion_probabilidades": {  
    "Bajo": 0.0421,  
    "Medio": 0.1023,  
    "Alto": 0.7843,  
    "Crítico": 0.0713  
  }  
}
```

Uso desde el Frontend

1. Navegar a /m1/ en el navegador
2. Completar el formulario "Predicción Individual de Riesgo"
3. Enviar para obtener la predicción
4. El resultado se muestra con color indicativo:
 - **Bajo:** Verde
 - **Medio:** Azul
 - **Alto:** Naranja
 - **Crítico:** Rojo

Pruebas de Validación

Caso 1 - Paciente Sano:

```
{"edad": 25, "IMC": 21.0, "glucosa": 85, "colesterol": 180,  
"presion_sistolica": 110, "presion_diastolica": 70,  
"frecuencia_cardiaca": 65, "fumador": 0}
```

→ **Resultado esperado:** Bajo con alta probabilidad

Caso 2 - Paciente Crítico:

```
{"edad": 70, "IMC": 38.0, "glucosa": 350, "colesterol": 290,  
"presion_sistolica": 190, "presion_diastolica": 110,  
"frecuencia_cardiaca": 105, "fumador": 1}
```

→ **Resultado esperado:** Crítico con alta probabilidad

5. Código del Predictor

El servicio RiskPredictor en `apps/ml/services/predictor.py` implementa:

```
class RiskPredictor:
    MAPEO_RIESGO = {'bajo': 0, 'medio': 1, 'alto': 2, 'critico': 3, 'crítico': 3}
    MAPEO_REVERSO = {0: 'Bajo', 1: 'Medio', 2: 'Alto', 3: 'Crítico'}
    FEATURES_COLS = ['IMC', 'edad', 'glucosa', 'colesterol',
                     'presion_sistolica', 'presion_diastolica',
                     'frecuencia_cardiaca', 'fumador']

    @classmethod
    def entrenar_modelo(cls):
        # Consulta ORM → DataFrame → Train/Test Split → Random Forest → Métricas → Guardar .pkl
        ...

    @classmethod
    def predecir_riesgo(cls, data):
        # Cargar modelo .pkl → Transformar input → Predecir clase + probabilidades
        ...
```

6. Endpoint de Entrenamiento

POST `/api/ml/entrenar/`

Response exitosa:

```
{
  "message": "Modelo entrenado y guardado correctamente.",
  "metrics": {
    "accuracy": 0.6389,
    "precision": 0.6394,
    "recall": 0.6389,
    "f1_score": 0.6347,
    "matriz_confusion": [[...]],
    "total_registros_entrenamiento": 1800
  }
}
```